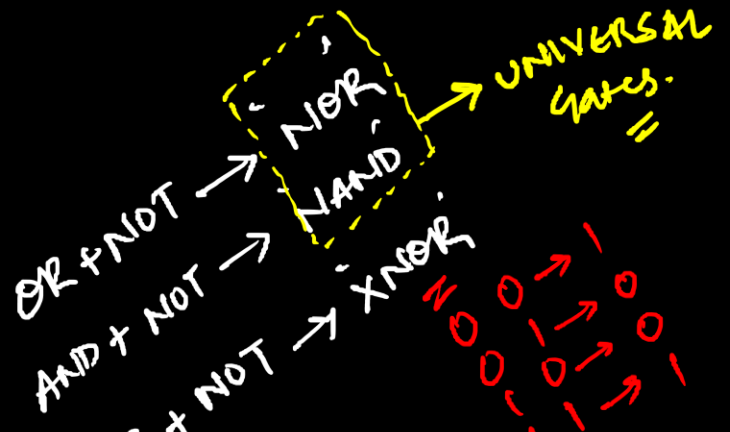


DAY 02 [VERILOG] : VLSI CLASSES Silicon [24-11-2024]

Sailesh Mishra



① VERILOG → Hardware Description Language. [HDL]
 → Synthesizable lang.
 → RTL Code (Register Transfer level).
 → Functional Verification.

Veri (VERIFY) + log (logic) = digital logic / Boolean algebra / Boolean logic

② "LOGIC" GATES → OR, AND, NOR, NAND, XOR, XNOR, NOT gate

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

③ COMBINATIONAL CKTS → FA ✓, HA ✓, FS ✓, HS ✓

SELECTION CKTS → MUX/DE-MUX, ENCODER/DE-CODER.

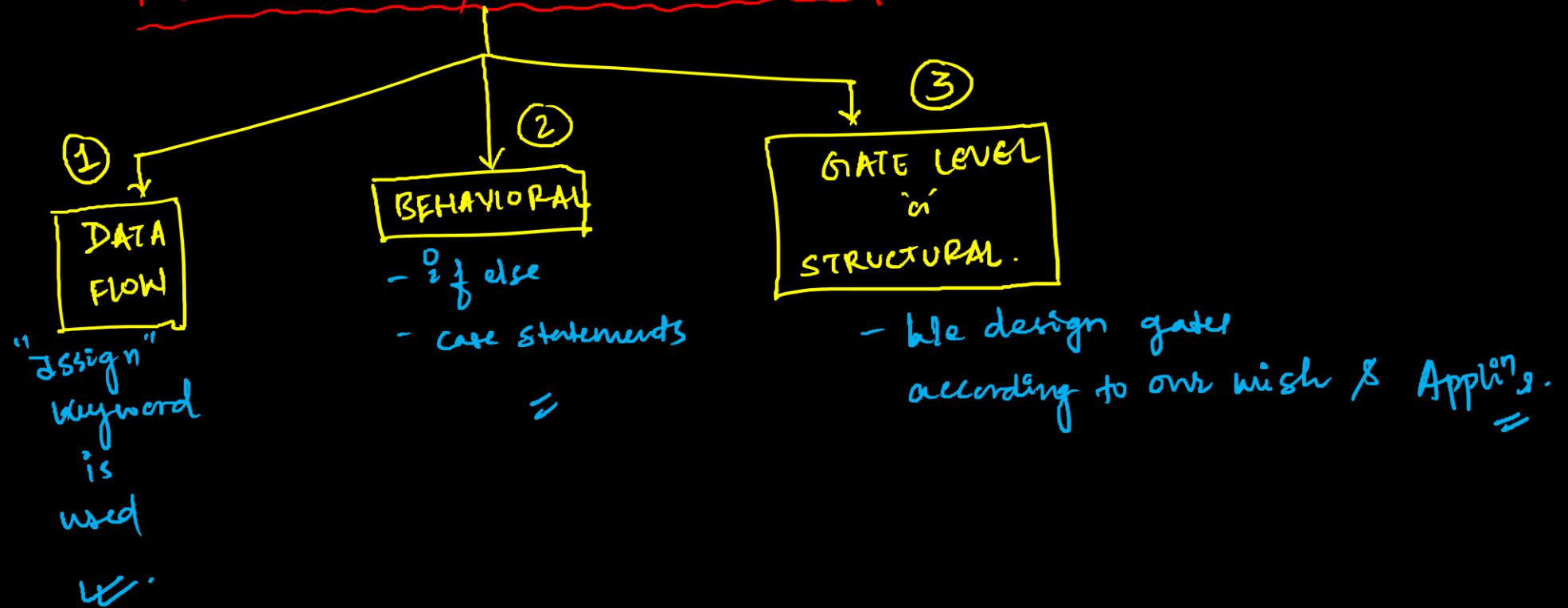
2:1, 4:1, 16:1 → 1:16, 1:4, 1:2

④ SEQUENTIAL CKTS → LATCHES → SR, D. ✓
 → FFs → SR, JK, MSJK, D, T

[finite state Machine] ✓
 ⑤ FSM DESIGN
 ⑥ ALU DESIGN

XOR gate is used as COMPARATOR Because; For Different i/p's; Y (o/p) = '1'.
 For Same i/p's; o/p = 0 (Hence, NO COMPARISON)

MODELLING TYPES/STYLES IN VERILOG



Brought to you by DOULOS

Languages & Libraries

Testbench + Design

SystemVerilog/Verilog

UVM / OVM

Other Libraries

Tools & Simulators

Cadence Xcelium 23.09

Compile Options

Run Options

Examples

using EDA Playground

VHDL

Verilog/SystemVerilog

UVM

EasierUVM

testbench.sv

```
1 //TESTBENCH → Breadband.
2 module and1_behav_tb;
3 wire t_c;
4 reg t_a, t_b;
5 and1_behav my_gate(t_a, t_b, t_c);
6 initial begin
7     $dumpfile("and1_behav_tb.vcd");
8     $dumpvars(1, and1_behav_tb);
9     $display("The INPUTS & OUTPUTS are : ");
10    $monitor("a = %d, b = %d, c = %d", t_a, t_b, t_c);
11    t_a = 1'b0; t_b = 1'b0;
12    #5
13    t_a = 1'b0; t_b = 1'b1;
14    #5
15    t_a = 1'b1; t_b = 1'b0;
16    #5
17    t_a = 1'b1; t_b = 1'b1;
18    #5 $finish;
19 end
20 endmodule
```

design.sv

```
1 // DESIGN
2 module and1_behav(a, b, c);
3 input a, b;
4 output reg c;
5 always@(a,b)
6 begin
7     if (a == 0 && b == 0) c <= 0;
8     else if (a == 0 && b == 1) c <= 0;
9     else if (a == 1 && b == 0) c <= 0;
10    else if (a == 1 && b == 1) c <= 1;
11 end
12 endmodule
```

→ to add module/ interfaces (.sv)

system verilog.

Breadband

Vertical rows (series s.c).

Horizontal rows (parallel s.c).

DIRECTED TESTBENCH.

Log Share

Top level design units:

and1_behav_tb

Loading snapshot worklib.and1_behav_tb:sv Done

xcelium> source /xcelium23.09/tools/xcelium/files/xmsimrc

xcelium> run

The INPUTS & OUTPUTS are :

a = 0, b = 0, c = 0

a = 0, b = 1, c = 0

a = 1, b = 0, c = 0

a = 1, b = 1, c = 1

Simulation complete via \$finish(1) at time 20 NS + 0

./testbench.sv:19 #5 \$finish;

xcelium> exit

TOOL: xrun 23.09-s001: Exiting on Nov 23, 2024 at 13:05:13 EST (total: 00:00:01)

Finding VCD file...

./and1_behav_tb.vcd

[2024-11-23 18:05:13 UTC] opening EPWave...

Done

meet.google.com is sharing your screen. Stop sharing Hide

EPWave

```

1 //TESTBENCH
2
3 module and1_behav_tb ;
4   wire t_c ;
5   reg t_a , t_b ;
6   and1_behav my_gate(t_a , t_b , t_c) ;
7   initial begin
8     $dumpfile("and1_behav_tb.vcd") ;
9     $dumpvars(1 , and1_behav_tb) ;
10    $display("The INPUTS & OUTPUTS are : ") ;
11    $monitor("a = %d , b = %d , c = %d" , t_a , t_b , t_c) ;
12    t_a = 1'b0 ; t_b = 1'b0 ;
13    #5
14    t_a = 1'b0 ; t_b = 1'b1 ;
15    #5
16    t_a = 1'b1 ; t_b = 1'b0 ;
17    #5
18    t_a = 1'b1 ; t_b = 1'b1 ;
19    #5 $finish;
20  end
21 endmodule

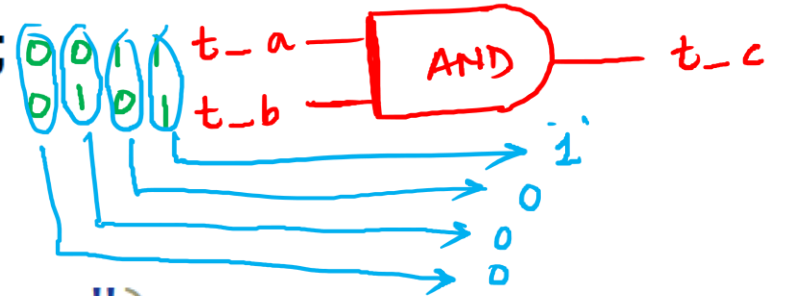
```

NO SPECIAL
CHARACTERS

MODULE NAME. (It should always end with an _tb)

Module Name always terminates with a Semi-colon (;)

DATA TYPES [In Verilog]
 { "wire" (volatile).
 "reg" (stores value).



WHOLE
MODULE

"MODULE INSTANTIATION" (same as function call).

Module Name

[and1_behav] ~
 { unit (unit under test).
 dut (design under test).

NOTE # 4 ⇒ [This will always be different from MODULE NAME.]

PROCEDURAL BLOCK → ① initial begin-end ✓
 → ② fork-join ✓
 → ③

NO DATATYPES are required

Only Pass the Variables.

→ including inputs, &
 → output.

→ irrespective of the datatype

6 [and1_behav] my_gate(t_a , t_b , t_c) ;

7 initial begin

8 \$dumpfile("and1_behav_tb.vcd") ;

→ file containing TIMING DIAGRAM of whatever's to
 You've wrote.

9 \$dumpvars(1 , and1_behav_tb) ;

→ MODULE NAME / ckt Name.

10 \$display("The INPUTS & OUTPUTS are : ") ;

// PRINTF STATEMENT...

11 \$monitor("a = %d , b = %d , c = %d" , t_a , t_b , t_c) ;

VERILOG OPERATORS

12 t_a = 1'b0 ; t_b = 1'b0 ;

13 #5

14 t_a = 1'b0 ; t_b = 1'b1 ;

15 #5

16 t_a = 1'b1 ; t_b = 1'b0 ;

17 #5

18 t_a = 1'b1 ; t_b = 1'b1 ;

19 #5 \$finish;

20 end

21 endmodule

→ So, whenever there'll be ANY CHANGE in the
 i/p & o/p ; \$monitor line will execute !!

(0-9) %d: DECIMAL
 (0-15) %h: HEXA-DEC.
 (0-3) %b: BINARY
 (0-7) %o: OCTAL

%d	%d	%d	%d
a = 0	b = 0	c = 0	
a = 0	b = 1	c = 0	
a = 1	b = 0	c = 0	
a = 1	b = 1	c = 1	

→ NO. of VARIABLES

(8) (3) → 0 to $2^n - 1 \Rightarrow$ OCTAL 0-2³-1 $\Rightarrow$ 0 to 7

12
13
14
15
16
17
18
19
20
21

`t_a = 1'b0 ; t_b = 1'b0 ;`

`#5` → DELAY → SI unit → pico second (10^{-12}): ps
→ nano second (10^{-9}): ns

`t_a = 1'b0 ; t_b = 1'b1 ;`

`#5` = 10ns

`t_a = 1'b1 ; t_b = 1'b0 ;`

`#5` = 15ns

`t_a = 1'b1 ; t_b = 1'b1 ;`

`#5` \$finish; // SIMULATION ENDS = 20ns/20ps.

end

endmodule

t_a t_b

0

0

0

1

1

0

1

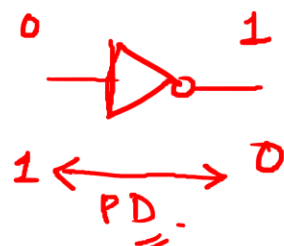
1



0	1 ✓
0	0.99 ✗

→ Accuracy

or
PRECISION



To decrease PROCESSING/COMPUTATIONAL Error

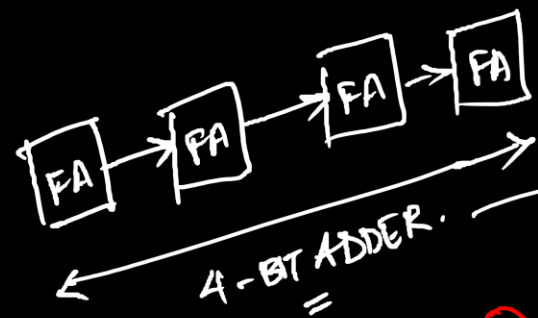
"DELAY" → to Make our DIGITAL SYSTEM STABLE
"STABILITY" Means Accurate/precise o/p

① BINARY $\rightarrow 2^0 \rightarrow 2^1 \rightarrow 2^2 \rightarrow 2^3 \dots$

0 1

3	2	1	0
8	4	2	1
	0	0	0
1	0	1	1

4-BIT ADDER
 $\downarrow$
 4 X (1-BIT Adders) ckt



② OCTAL $\rightarrow 8 \rightarrow 2^3$

$$\begin{array}{r} 1011 \text{ (11)} \\ + 0101 \text{ (5)} \\ \hline 10000 \end{array}$$
 $2^4 = 16$

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

③ HEXA-DECIMAL $\rightarrow 16 \rightarrow 2^4$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	10
1	0	1	1	11
1	1	0	0	12
1	1	0	1	13
1	1	1	0	14
1	1	1	1	15

1 Bit adder $\rightarrow A, B$



- $10 (A) \rightarrow 1010$
 $11 (B) \rightarrow 1011$
 $12 (C) \rightarrow 1100$
 $13 (D) \rightarrow 1101$
 $14 (E) \rightarrow 1110$
 $15 (F) \rightarrow 1111$

1 // DESIGN

2
3 module and1_behav(a, b, c);

4 input a, b;

5 output reg c;

6 always@(a, b)
7 begin

8 if (a == 0 && b == 0) c <= 0;

9 else if (a == 0 && b == 1) c <= 0;

10 else if (a == 1 && b == 0) c <= 0;

11 else if (a == 1 && b == 1) c <= 1;

12 end

13 endmodule

14

INSTANTIATED
MODULE which is called/invoked in the TESTBENCH side

ONLY VARIABLES; NO
Datatypes.

Under SENSITIVITY LIST, only those variables can stay
whose value are to CHANGE !!

Whatever written inside the BRACKET Comes under the
SENSITIVITY LIST !! i.e. 'a' & 'b'

ALWAYS
BLOCK.